

GREI - USC

BASES DE DATOS II

Resumen conocimientos teóricos

Ismael Castiñeira Vázquez

ismael.castineira@rai.usc.es

Programación con SQL

1. ¿Por qué no es SQL un lenguaje computacionalmente completo? ¿Qué soluciones conoces para aumentar las capacidades de cómputo de SQL?

SQL no es un lenguaje computacionalmente completo ya que carece de **recursividad** y **estructuras repetitivas** (bucles). No obstante, a partir de SQL 2003 se introdujo la recursividad aunque de forma limitada. En cambio, los bucles se pueden conseguir pero combinando SQL con lenguajes procedimentales.

2. SQL embebido: Describe su funcionamiento. Limitaciones de SQL embebido estático.

El **SQL embebido** es aquel en el que el código SQL se mezcla con el código del lenguaje anfitrión. Existe un proceso de precompilación que transforma el SQL en llamadas a rutinas al gestor.

El **SQL embebido estático** presenta ciertas **limitaciones** que el SQL embebido dinámico soluciona. Estas son:

- Generar aplicaciones en las que el usuario pueda crear de forma interactiva sus propias consultas.
- Consultas que no sean estáticas. [**SELECT** * **FROM** :NombreTabla]
- Necesidad de utilizar variables en nombres de tablas y de atributos de tablas.
- Problema para utilizar FETCH si no se conocen los nombres de los atributos en tiempo de compilación.

3. Librerías de acceso a datos: ¿Qué tipo de funcionalidad proporcionan sus funciones? ¿Qué problema existe con el uso de las librerías de PHP? Soluciones propuestas con ODBC y JDBC.

Las **librerías de acceso a datos** presentan un conjunto de funciones del lenguaje anfitrión para establecer conexiones, ejecutar consultas, obtener y procesar resultados, gestionar errores y otras funciones que SQL no permite hacer.

El principal problema que existe con PHP es que al cambiar de SGBD tienes que **cambiar las funciones de conexión a la base de datos**, mientras que con ODBC y JDBC esto no ocurre. Tanto **ODBC** como **JDBC** tienen una capa de software intermedia entre la base de datos y la aplicación final que se conecta a la base de datos, de forma que **si se cambia la base de datos o el SGBD no es necesario cambiar el código de la aplicación**, ya que utiliza las mismas funciones disponibles en la librería de acceso a datos.

Gestión de transacciones

4. ¿Qué es una transacción? Pon un ejemplo. Describe las propiedades ACID de las transacciones detallando qué o quién debe de ser el responsable de su cumplimiento.

Una **transacción** es una unidad de la ejecución de un programa que accede y posiblemente actualiza varios elementos de datos. Una transacción se inicia por la ejecución de un programa de usuario escrito en un lenguaje de manipulación de datos de alto nivel o en un lenguaje de programación y está delimitado por instrucciones de la forma **inicio transacción** y **fin transacción**. La transacción consiste en todas la operaciones que se ejecutan entre inicio transacción y el fin transacción.

```
Ti leer(A);  
    A := A-50;  
    escribir(A);  
    leer(B)  
    B:=B+50;  
    escribir(B);
```

Para asegurar la integridad de los datos se necesita que el sistema de base de datos mantenga las siguientes propiedades de las transacciones:

- **Atomicidad:** O todas las operaciones de la transacción se realizan adecuadamente en la base de datos o ninguna de ellas. Es responsabilidad del SGBD, concretamente del componente de gestión de transacciones.
- **Consistencia:** La ejecución aislada de la transacción conserva la consistencia de la base de datos. Es responsabilidad del programador.
- **Aislamiento:** Esta propiedad asegura que una T_i no puede afectar a una T_j . De esta forma se asegura que dos transacciones sobre la misma información sean independientes y no generen ningún tipo de error. Es responsabilidad del SGBD, concretamente del componente de control de concurrencia.
- **Durabilidad:** Tras la finalización con éxito de una transacción, los cambios realizados en la base de datos permanecen, incluso si hay fallos en el sistema. Es responsabilidad del SGBD, concretamente del componente de gestión de recuperaciones.

5. Describe los estados por los que puede pasar la ejecución de una transacción. Pon un ejemplo. Comenta el problema de las escrituras externas observables.

Una transacción debe estar en uno de los estados siguientes:

- **Activa:** el estado inicial; la transacción permanece en este estado durante su ejecución.
- **Parcialmente comprometida:** después de ejecutarse la última instrucción.
- **Fallida:** tras descubrir que no puede continuar la ejecución normal.
- **Abortada:** después de haber retrocedido la transacción y restablecido la base de datos a su estado anterior al comienzo de la transacción.
- **Comprometida:** tras completarse con éxito.

Hay que tener cuidado cuando se trabaja con **escrituras externas observables**, como en un terminal o una impresora. Cuando una escritura así tiene lugar, no puede borrarse puesto que puede haber sido vista fuera del sistema de base de datos. Muchos sistemas permiten que tales escrituras tengan lugar sólo después de que la transacción llegue al estado Comprometida. Una manera de implementar dicho esquema es hacer que el sistema de base de datos almacene temporalmente cualquier valor asociado con estas escrituras externas en memoria no volátil, y realice

las escrituras actuales sólo si la transacción llega al estado Comprometida. Si el sistema falla después de que la transacción llegue al estado Comprometida, pero antes de que finalicen las escrituras externas, el sistema de base de datos puede llevar a cabo dichas escrituras externas cuando el sistema se reinicie.

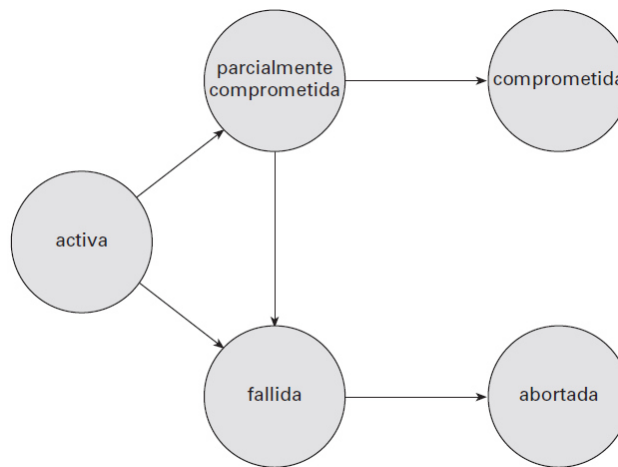


Diagrama de transición de estado de una transacción.

Para algunas aplicaciones puede ser deseable permitir a las transacciones activas que muestren datos a los usuarios, particularmente para transacciones de larga duración que se ejecutan durante minutos u horas.

6. Implementación de la atomicidad y de la durabilidad: Describe la solución de copia en la sombra destacando sus ventajas y desventajas. ¿Qué otras soluciones conoces?

El componente de gestión de recuperaciones de un sistema de base de datos implementa el soporte para la atomicidad y la durabilidad. Consideraremos un esquema simple pero extremadamente ineficiente, denominado **copia en la sombra**. Este esquema asume que sólo una transacción está activa en cada momento.

Con este esquema, cuando una transacción quiere actualizar la base de datos, crea primero una copia completa de dicha base de datos. Todos los cambios se hacen en la nueva copia de la base de datos dejando la copia original, la copia **en la sombra**, inalterada. Si en cualquier punto hay que abortar la transacción, la copia nueva simplemente se borra. La copia antigua de la base de datos no se ve afectada.

Si la transacción se completa, se compromete como sigue; la nueva copia se convierte entonces en la copia de la base de datos actual y la copia antigua de la base de datos se borrará después.

Esta implementación es extremadamente ineficiente en el contexto de grandes bases de datos, ya que la ejecución de una simple transacción requiere copiar la base de datos completa. Además, la implementación no permite a las transacciones ejecutarse concurrentemente unas con otras. Existen formas prácticas de implementar la atomicidad y durabilidad que son mucho menos costosas y más potentes como **esquemas basados en registro histórico**, esquemas **de modificación diferida** o **esquemas de modificación inmediata**.

7. Secuencialidad: ¿Cuándo es una planificación secuenciable? ¿Cuándo son dos planificaciones equivalentes en cuanto a conflictos? ¿Cuándo es una planificación secuenciable en cuanto a conflictos?

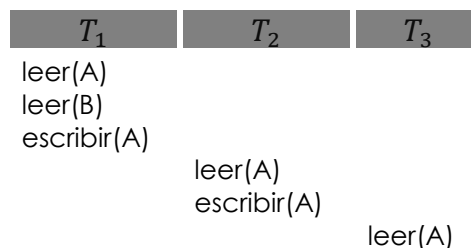
Una **planificación** para un conjunto de transacciones es el orden en el que se realizan los pasos elementales de las transacciones. Una planificación es **secuencial** si todos los pasos de cada transacción ocurren consecutivamente. Una planificación es **secuenciable** si su efecto es equivalente al de la planificación secuencial.

Si una planificación P se puede transformar en otra P' por medio de una serie de intercambios de instrucciones no conflictivas, se dice que P y P' son **equivalentes en cuanto a conflictos**.

Se dice que una planificación P es **secuenciable en cuanto a conflictos** si es equivalente en cuanto a conflictos a una planificación secuencial.

8. ¿Cuándo no es una planificación recuperable? ¿Cuándo presenta una planificación retrocesos en cascada?

Una **planificación no recuperable** es aquella en la que para todo par de transacciones T_i y T_j tales que T_j lee elementos de datos que ha escrito previamente T_i , la operación comprometer de T_i no aparece antes que la de T_j .



Ejemplo de planificación.

A pesar de que una planificación sea recuperable, puede que haya que retroceder varias transacciones para recuperar correctamente el estado previo a un fallo en una transacción T_i . Tales situaciones ocurren si las transacciones leen datos que ha escrito T_i . Este fenómeno en el cual un fallo en una única transacción provoca una serie de retrocesos de la transacción se denomina **retroceso en cascada**.

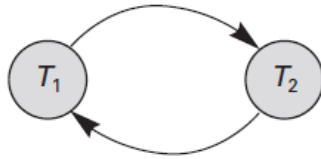
9. Describe alguna técnica que conozcas para comprobar si una planificación es secuenciable en cuanto a conflictos.

Considérese una planificación P . Se construye un grafo dirigido, llamado **grafo de precedencia** para P . Este grafo consiste en un par $G = (V, A)$, siendo V un conjunto de vértices y A un conjunto de arcos. El conjunto de vértices consiste en todas las transacciones que participan en la planificación. El conjunto de arcos consiste en todos los arcos $T_i \rightarrow T_j$ para los cuales se dan una de las tres condiciones siguientes:

- T_i ejecuta *escribir*(Q) antes de que T_j ejecute *leer*(Q).
- T_i ejecuta *leer*(Q) antes de que T_j ejecute *escribir*(Q).
- T_i ejecuta *escribir*(Q) antes de que T_j ejecute *escribir*(Q).

Si existe un arco $T_i \rightarrow T_j$ en el grafo de precedencia, entonces en toda planificación secuencial P' equivalente a P , T_i debe aparecer antes de T_j .

Si el grafo de precedencia de P tiene un ciclo, entonces la planificación P **no es secuenciable en cuanto a conflictos**. Si el grafo no tiene ciclos, entonces la planificación P es **secuenciable en cuanto a conflictos**.



Ejemplo de grafo de precedencia para dos transacciones T_1 y T_2 .

Control de concurrencia

10. *Protocolos basados en bloqueos: Describe lo que son los bloqueos y la compatibilidad entre ellos. ¿A qué componente del SGBD se le piden? ¿Es posible usar bloqueos y tener planificaciones no secuenciables? ¿Por qué? ¿Qué es un interbloqueo? ¿Qué es un protocolo? ¿Qué es una planificación legal para un protocolo? ¿Cuándo podemos decir que un protocolo asegura la Secuencialidad en cuanto a conflictos (grafo de precedencia)? ¿Cuándo se produce la inanición? ¿Cómo se puede evitar?*

Una forma de asegurar la secuencialidad es exigir que el acceso a los elementos de datos se haga en exclusión mutua; es decir, mientras una transacción accede a un elemento de datos, ninguna otra transacción pueda modificar dicho elemento. El método más habitual que se usa para implementar este requisito es permitir que una transacción acceda a un elemento de datos sólo si posee actualmente un **bloqueo** sobre dicho elemento. La petición de los bloqueos se hace al **gestor de control de concurrencia**.

Existen muchos modos mediante los cuales se puede bloquear un elemento de datos. Dado un conjunto de modos de bloqueo, se puede definir sobre ellos una función de **compatibilidad** como sigue. Utilizamos A y B para representar dos modos de bloqueo arbitrarios. Supóngase que la transacción T_i solicita un bloqueo en modo A sobre el elemento Q , en el que la transacción T_j ($T_i \neq T_j$) posee actualmente un bloqueo de modo B . Si la transacción T_i se le puede conceder un bloqueo sobre Q a pesar de la presencia del bloqueo de modo B , entonces se dice que el modo A es compatible con el modo B .

Es posible utilizar bloqueos y tener **planificaciones no secuenciables**, sólo si las transacciones liberan los recursos que poseen justo cuando acaban de utilizarlos, y no cuando se comprometen.

El uso de bloqueos puede conducir a una situación no deseada. Cuando dos transacciones se encuentran una a la espera de que la otra libere un bloqueo para poder continuar se entra en una situación denominada **interbloqueo**.

Se exige que toda transacción del sistema siga un conjunto de reglas llamado **protocolo de bloqueo**, que indica el momento en que una transacción puede bloquear y desbloquear cada uno de los elementos de datos. Se dice que una planificación S es **legal** bajo un protocolo de bloqueo dado si S es una planificación posible para un conjunto de transacciones que sigan las reglas del protocolo de bloqueo. Se dice que un protocolo **asegura la secuencialidad en cuanto a conflictos** si y sólo si todas las planificaciones legales son secuenciables en cuanto a conflictos; en otras palabras, para todas las planificaciones legales la relación $\rightarrow$ asociada es acíclica.

Puede ocurrir el caso de que exista una secuencia de transacciones que soliciten un bloqueo en modo compartido sobre el elemento Q y que cada ellas libere el bloqueo un poco después de que sea concedido, de forma que T_i (que suponemos que había solicitado un bloque en modo exclusivo por lo que debe esperar a que las demás transacciones que tienen el elemento Q en modo compartido lo liberen) nunca obtenga el bloque en modo exclusivo sobre Q . La transacción T_i nunca progresa y se dice que tiene **inanición**.

Se puede **evitar la inanición** de las transacciones al conceder los bloqueos de la siguiente forma: cuando una transacción T_i solicita un bloqueo sobre un elemento de

datos Q en un modo particular M , el gestor de control de concurrencia concede el bloqueo siempre que:

- No exista otra transacción que posea un bloqueo sobre Q en un modo que esté en conflicto con M .
- No exista otra transacción que esté esperando un bloqueo sobre Q y que lo haya solicitado antes de T_i .

De este modo, una petición de bloqueo nunca se quedará bloqueada por otra petición de bloqueo solicitada más tarde.

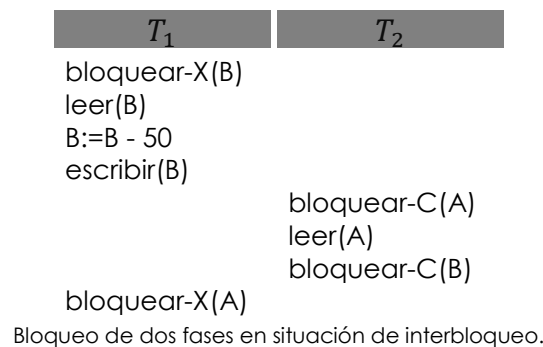
11. Protocolo de bloqueo en dos fases. Variantes: estricto y riguroso. ¿Cómo se puede aumentar la concurrencia? ¿Puede el sistema generar los bloqueos de forma automática, o debe intervenir el usuario?

Un protocolo que asegura la secuencialidad es el **protocolo de bloqueo de dos fases**. Este protocolo exige que cada transacción realice las peticiones de bloqueo y desbloqueo de dos fases:

- Fase de crecimiento:** Una transacción puede obtener bloqueos pero no puede liberarlos.
- Fase de decrecimiento:** Una transacción puede liberar bloqueos pero no puede obtener ninguno nuevo.

Inicialmente una transacción está en la fase de crecimiento. La transacción adquiere los bloqueos que necesite. Una vez que la transacción libera un bloqueo, entra en la fase de decrecimiento y no puede realizar más peticiones de bloqueo.

El protocolo de bloque de dos fases asegura la secuencialidad en cuanto a conflictos pero no asegura la ausencia de interbloqueos. El retroceso en cascada puede ocurrir en el bloqueo de dos fases.



Los retrocesos en casca se pueden evitar por medio de una modificación del protocolo de bloqueo de dos fases que se denomina **protocolo de bloqueo estricto de dos fases**. Este protocolo exige que una transacción debe poseer todos los bloqueos en modo exclusivo que tome hasta que dicha transacción se complete.

Otra variante del bloqueo de dos fases es el **protocolo de bloqueo riguroso de dos fases**, el cual exige que se posean todos los bloqueos hasta que se comprometa la transacción. Con esta variante se pueden secuenciar las transacciones en el orden en el que se comprometen.

El aumento **de la concurrencia** lleva a un refinamiento del protocolo de bloqueo de dos fases básico en el cual se permiten **conversiones de bloqueo**. Se va a

proporcionar un mecanismo para cambiar un bloqueo compartido por uno exclusivo (**subir**) y uno exclusivo por uno compartido (**bajar**). No se permite la conversión de modos arbitrariamente. La subida puede tener lugar en la fase de crecimiento y la bajada en la fase de decrecimiento.

Existe un mecanismo llamado **gestor de bloqueos** que se implementa como un proceso que recibe mensajes de transacciones y envía mensajes como respuesta. De esta forma **evitamos la intervención del usuario**.

12. Prevención de los interbloqueos. Describe las dos soluciones que conoces.

El esquema más simple para la **primera aproximación** exige que cada transacción bloquee todos sus elementos de datos antes de comenzar su ejecución. Además, o bien se bloquean todos en un paso o ninguno de ellos se bloquea. Este protocolo tiene dos inconvenientes principales:

- A menudo es difícil predecir, antes de que comience la transacción, cuáles son los elementos de datos que es necesario bloquear.
- La utilización de elementos de datos puede ser muy baja, ya que muchos de los elementos de datos pueden estar bloqueados pero sin usar durante mucho tiempo.

Otro esquema para prevenir interbloqueos consiste en imponer un orden parcial a todos los elementos de datos y exigir que una transacción bloquee un elemento de datos sólo en el orden que especifica dicho orden parcial.

La **segunda aproximación** para prevenir interbloqueos consiste en utilizar expropiación y retrocesos de transacciones. En la expropiación, cuando la transacción T_2 solicita un bloqueo que la transacción T_1 posee, el bloqueo concedido a T_1 debe expropiarse retrocediendo T_1 y concediéndolo a continuación a T_2 . Para controlar la expropiación se asigna una marca temporal única a cada transacción. El sistema usa estas marcas temporales sólo para decidir si una transacción debe esperar o retroceder. Para el control de concurrencia se sigue usando el bloqueo. Si una transacción se retrocede mantiene su marca temporal antigua cuando vuelva a comenzar. Se han propuesto dos esquemas de prevención de interbloqueos que usan marcas temporales:

- Esquema **esperar-morir**: está basado en una técnica sin expropiación. Cuando la transacción T_i solicita un elemento de datos que posee actualmente T_j , T_i puede esperar sólo si tiene una marca temporal más pequeña que la de T_j . En otro caso T_i se retrocede (muere)
- Esquema **herir-esperar**: está basado en una técnica de expropiación. Es el opuesto del esquema esperar-morir. Cuando la transacción T_i solicita un elemento de datos que posee actualmente T_j , T_i puede esperar sólo si tiene una marca temporal mayor que la de T_j . En otro caso T_j se retrocede (T_i hiere a T_j).

Siempre que se retrocedan las transacciones, es importante asegurarse de que no exista **inanición**, es decir, ninguna transacción se retrocede repetidamente y nunca se le permite progresar. El principal problema de cualquiera de estos dos esquemas es que se pueden dar **retrocesos** innecesarios

13. Detección y recuperación de los interbloqueos.

Si el sistema no utiliza algún protocolo que asegure la ausencia de interbloqueos, entonces se debe usar un **esquema de detección y recuperación**. Periódicamente se invoca a un algoritmo que examina el estado del sistema para determinar si hay un interbloqueo. Si lo hay, entonces el sistema debe intentar recuperarse del interbloqueo. Para ello, el sistema debe:

- Mantener información sobre la asignación de los elementos de datos a las transacciones, así como de toda petición de elemento de datos pendiente.
- Proporcionar un algoritmo que use esta información para determinar si el sistema ha entrado en un estado de interbloqueo.
- Recuperarse del interbloqueo cuando el algoritmo de detección determine que existe un interbloqueo.

Detección de interbloqueos

Los interbloqueos se pueden describir con precisión por medio de un grafo dirigido llamado **grafo de espera**. Este grafo consiste en un par $G = (V, A)$, siendo V el conjunto de vértices y A el conjunto de arcos. El conjunto de vértices consiste en todas las transacciones del sistema. Cada elemento del conjunto E de arcos es un par ordenado $T_i \rightarrow T_j$. Si $T_i \rightarrow T_j$ pertenece a A entonces hay un arco dirigido de la transacción T_i a T_j , lo cual implica que la transacción T_i está esperando a que la transacción T_j libere un elemento de datos que necesita.

Existe un interbloqueo en el sistema si y sólo si el grafo de espera contiene un ciclo, por lo que toda transacción involucrada en el ciclo está interbloqueada. Para detectar los interbloqueos el sistema debe mantener el grafo de espera y debe invocar periódicamente a un algoritmo que busque un ciclo en el grafo. Si los interbloqueos ocurren con frecuencia, entonces se debe invocar al algoritmo de detección con mayor frecuencia de la usual.

Recuperación de interbloqueos

Cuando un algoritmo de detección de interbloqueos determina que existe un interbloqueo, el sistema debe recuperarse del mismo. La solución más común es retroceder una o más transacciones para romper el interbloqueo. Se deben realizar tres acciones:

- a. Selección de una víctima:** Dado un conjunto de transacciones interbloqueadas se debe determinar la transacción (o transacciones) que se van a retroceder, que serán aquellas que incurran en un coste mínimo. Hay muchos factores que determinan el coste de un retroceso, como:
 - o Lo que la transacción ha calculado y lo que le queda.
 - o El número de elementos de datos que ha usado la transacción.
 - o La cantidad de datos que necesita la transacción para que se complete.
 - o El número de transacciones que se verán involucradas en el retroceso.
- b. Retroceso:** Se debe determinar hasta dónde se retrocederá la transacción. La solución más simple consiste en un **retroceso total**: se aborta la transacción y luego vuelve a comenzar. No obstante, es más efectivo retroceder la transacción sólo lo necesario para romper el interbloqueo. Dicho **retroceso parcial** requiere que el sistema mantenga información adicional sobre el estado de todas las transacciones que están en ejecución.

- c. **Inanición:** al ser un sistema basado en factores de coste, puede ocurrir que siempre se elija a la misma víctima. Se soluciona incluyendo en el factor de coste el número de retrocesos.

14. Operaciones de inserción y borrado. El fenómeno fantasma. Soluciones para el fenómeno fantasma.

Algunas transacciones necesitan no sólo acceder a los elementos de datos existentes, sino también poder crear nuevos elementos de datos o tener la posibilidad de borrarlos. Para examinar la forma en que tales transacciones afectan al control de concurrencia se introducen las operaciones de **borrado** e **inserción**.

Borrado

Se debe decidir en qué casos una instrucción **borrar** está en conflicto con otra instrucción. Sean I_i e I_j instrucciones de T_i y T_j , respectivamente, que están consecutivas en la planificación S . Sea $I_i = \text{borrar}(Q)$. Se consideran distintas instrucciones I_j :

- $I_j = \text{leer}(Q)$. Están en conflicto si I_i está antes de I_j habrá un error lógico, en caso contrario T_j , ejecutará con éxito su operación.
- $I_j = \text{escribir}(Q)$. Igual que en el caso anterior.
- $I_j = \text{borrar}(Q)$. Habrá error en cualquier caso.
- $I_j = \text{insertar}(Q)$. Están en conflicto. Si no existe Q previamente I_i primero producirá error. Si existe Q e I_j está primero también habrá error lógico.

Se puede concluir lo siguiente:

- En el protocolo de dos fases se necesita un bloqueo exclusivo en un elemento de datos antes de que se borre dicho elemento.
- En el protocolo de ordenación por marcas temporales se debe hacer una prueba similar que la que se hacía con escribir.

Inserción

La instrucción **insertar**(Q) está en conflicto con las operaciones **leer**(Q) y **escribir**(Q). No se puede realizar operaciones sobre un elemento de datos hasta que este último exista.

Puesto que **insertar**(Q) asigna un valor al elemento de datos Q , se trata **insertar** de forma similar a **escribir** desde el punto de vista del control de concurrencia:

- En el protocolo de bloqueo de dos fases si T_i realiza la operación **insertar**(Q) se da a T_i un bloqueo exclusivo sobre Q .
- En el protocolo de ordenación por marcas temporales se fijan los valores $\text{marca} - \text{temporal} - L(Q)$ y $\text{marca} - \text{temporal} - E(Q)$ a $MT(T_i)$.

El fenómeno fantasma

Las inserciones pueden dar lugar al **fenómeno fantasma**, un conflicto entre una inserción y una consulta incluso si las dos transacciones no acceden a tuplas comunes. Tales conflictos no se pueden detectar si el bloqueo se ha hecho sólo sobre las tuplas a las que han accedido las transacciones. Se necesita bloquear la información acerca de qué tuplas pertenecen a la relación.

La **solución más sencilla** consiste en asociar un elemento de datos con la propia relación. Éste representa la información utilizada para encontrar las tuplas en la relación. Este elemento de datos se bloqueará y así se evita que otras transacciones actualicen el valor acerca de qué tuplas pertenecen a la relación. El inconveniente principal de esta solución es el bajo nivel de concurrencia.

Una solución mejor es la **técnica de bloqueo de índice**. Toda transacción que inserte una tupla en una relación debe insertar información en cada uno de los índices que se mantenga en la relación. Esta solución exige bloqueos sobre ciertos cajones de índices. Estos bloqueos aseguran que todas las transacciones conflictivas están en conflicto por un elemento de datos real en lugar de por uno fantasma.

15. Niveles débiles de consistencia: Grado dos, estabilidad del cursor. Niveles débiles de consistencia en SQL: 1992.

Consistencia de grado dos

El objetivo es **evitar abortar en cascada** sin asegurar necesariamente la secuencialidad. El protocolo de bloqueo para la consistencia de grado dos utiliza los mismos modos de bloqueo que se utilizan para el protocolo de bloqueo de dos fases: compartido (C) y exclusivo (X).

A diferencia de la situación en los bloqueos de dos fases, los bloqueos-C **pueden liberarse en cualquier momento** así como se pueden **establecer bloqueos en cualquier momento**. Los bloqueos exclusivos no se pueden liberar hasta que la transacción se comprometa o se aborte. La secuencialidad no está asegurada por este protocolo.

La posibilidad de que se produzca inconsistencia con la consistencia de grado dos hace que este enfoque no sea conveniente para muchas aplicaciones.

Estabilidad del cursor

Es una forma de consistencia de grado dos diseñada para programas escritos en lenguajes de propósito general, los cuales **iteran sobre las tuplas de una relación utilizando cursores**. En lugar de bloquear toda la relación, la estabilidad del cursor asegura que:

- La tupla que está procesando la iteración esté bloqueada en modo compartido.
- Todas las tuplas modificadas estén bloqueadas en modo exclusivo hasta que se comprometa la transacción.

Niveles débiles de consistencia en SQL

La norma SQL también permite que una transacción especifique si puede ser ejecutada de tal forma que se convierta en no secuenciable con respecto a otras transacciones.

Los niveles de consistencia especificados por SQL-92 son los siguientes:

- **Secuenciable:** Es el predeterminado.
- **Lectura repetible:** Sólo permite leer registros comprometidos, y además requiere que, entre dos lecturas de un registro realizadas por una transacción, no se permita que ninguna otra transacción actualice el registro. Sin embargo, la transacción puede no ser secuenciable con respecto a otras transacciones.

- **Compromiso de lectura:** sólo permite leer registros comprometidos, pero ni siquiera requiere lecturas repetibles.
- **Sin compromiso de lectura:** permite incluso leer registros no comprometidos. Éste es el nivel de consistencia más bajo que permite SQL-92.

Sistema de recuperación

16. Estructura del almacenamiento: Tipos de almacenamiento. Implementación del almacenamiento estable. Acceso a los datos.

Tipos de almacenamiento

Para comprender cómo se pueden garantizar las propiedades de atomicidad y durabilidad de una transacción, se deben comprender mejor estos medios de almacenamiento y sus métodos de acceso. Los distintos **medios de almacenamiento** son los siguientes:

- a. **Almacenamiento volátil:** La información que reside en él no suele sobrevivir a las caídas del sistema. El acceso al almacenamiento es muy rápido. (Memoria caché y memoria principal).
- b. **Almacenamiento no volátil:** La información que reside en él sobrevive a las caídas del sistema. Los discos y cintas magnéticas son ejemplos de este almacenamiento. Los discos se utilizan para almacenamiento en conexión, mientras que las cintas se usan para almacenamiento permanente. A pesar de ellos, ambos pueden fallar, lo que puede conducir a una pérdida de información. Es más lento que el almacenamiento volátil.
- c. **Almacenamiento estable:** La información que reside en almacenamiento estable nunca se pierde.

Implementación del almacenamiento estable

Para implementarla se debe replicar la información necesaria en varios medios de almacenamiento no volátil (discos) con modos de fallo independientes, y actualizar esa información de manera controlada para asegurar que un fallo durante una transferencia de datos no dañará la información necesaria.

Los sistemas más seguros guardan una copia de cada bloque de almacenamiento estable en un lugar remoto, escribiéndola a través de una red de computadoras, además de almacenar el bloque en un sistema de discos locales.

Una transferencia de bloques entre la memoria y el disco puede acabar de diferentes formas:

- **Éxito:** la información transferida llega a su destino con seguridad.
- **Fallo parcial:** Ocurre un fallo en medio de la transferencia y el bloque de destino contiene información incorrecta.
- **Fallo total:** El fallo ocurre suficientemente pronto durante la transferencia para que el bloque de destino permanezca intacto.

Si se produce un **fallo durante una transferencia de datos**, es necesario que el sistema lo detecte e invoque a un procedimiento de recuperación para restaurar el bloque a un estado estable. Una operación de salida se ejecuta de la siguiente manera:

- Se escribe la información en el primer bloque físico.
- Cuando la primera escritura se completa con éxito, se escribe la misma información en el segundo bloque físico.
- La salida está completada sólo después de que la segunda escritura finalice con éxito.

Durante la **recuperación** se examina cada par de bloques físicos. Si ambos coinciden y no existe ningún error detectable, entonces no son necesarias más

acciones. Si un bloque contiene un error detectable, se reemplaza su contenido por el del segundo bloque.

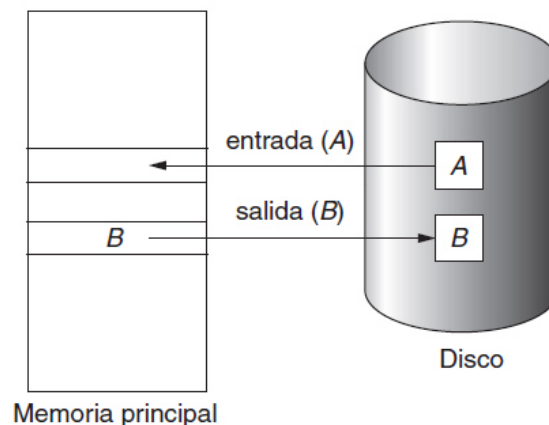
Acceso a los datos

El sistema de bases de datos reside permanentemente en almacenamiento no volátil y se divide en unidades de almacenamiento de longitud fija denominada bloques.

Las transacciones llevan información del disco hacia la memoria principal y luego devuelven la información al disco. Las operaciones de entrada y salida se realizan en unidades de bloque. Nos referiremos a los bloques que residen en el disco como **bloques físicos**, y a los que residen temporalmente en la memoria principal como **bloques de memoria intermedia**. El área de memoria en donde los bloques residen temporalmente se denomina **memoria intermedia de disco**.

Las transferencias de un bloque entre disco y memoria principal se comienzan a través de las dos operaciones siguientes:

- entrada(B)**: transfiere el bloque físico B a la memoria principal.
- salida(B)**: transfiere el bloque de memoria intermedia B al disco y reemplaza allí al correspondiente bloque físico.



Operaciones de almacenamiento de bloques.

Cada transacción T_i posee un área de trabajo privada en la cual se guardan copias de todos los elementos de datos accedidos y actualizados por T_i . Cada elemento de datos X almacenado en el área de trabajo de la transacción T_i se denotará como x_i . Se realizan transferencias de datos utilizando las dos operaciones siguientes:

- leer(X)**: Asigna el valor del elemento de datos X a la variable local x_i .
 - Si el bloque B_x en el que reside X no está en la memoria principal, entonces se emite **entrada(B_x)**.
 - Asignar a x_i el valor de X en el bloque de memoria intermedia.
- escribir(X)**: Asigna el valor de la variable local x_i al elemento de datos X en el bloque de memoria intermedia.
 - Si el bloque B_x en el que reside X no está en la memoria principal, entonces se lanza **entrada(B_x)**.
 - Asignar el valor de x_i a X en la memoria intermedia B_x .

Un bloque de memoria intermedia B se escribe finalmente en el disco bien porque el gestor de la memoria intermedia necesita espacio en memoria para otros

propósitos, o bien porque el sistema de base de datos desea reflejar en el disco los cambios sufridos por B . Diremos que el sistema de bases de datos **fuera la salida** de la memoria intermedia B si se ejecuta la orden $salida(B)$.

17. Recuperación basada en el registro histórico. Modificación diferida. Modificación inmediata. Puntos de revisión.

El registro histórico es una secuencia de **registros** que mantiene un registro de todas las actividades de actualización de la base de datos. Un **registro de actualización del registro histórico** describe una única escritura en la base de datos y tiene los siguientes campos:

- El **identificador de la transacción**: es un identificador único de la transacción que realiza la operación *escribir*.
- El **identificador del elemento de datos**: es un identificador único del elemento de datos que se escribe. Normalmente suele coincidir con la ubicación del elemento de datos en el disco.
- El **valor anterior**: es el valor que tenía el elemento de datos antes de la escritura.
- El **valor nuevo** es el valor que tendrá el elemento de datos después de la escritura.

Existen **otros registros** del registro histórico especiales para registrar sucesos significativos durante el procesamiento de una transacción:

- $\langle T_i \text{ iniciada} \rangle$: La transacción T_i ha comenzado.
- $\langle T_i, X_j, V_1, V_2 \rangle$: La transacción T_i ha realizado una escritura sobre el elemento de datos X_j . X_j tenía el valor V_1 antes de la escritura y tendrá el valor V_2 después de la escritura.
- $\langle T_i \text{ comprometida} \rangle$: La transacción T_i se ha comprometido.
- $\langle T_i \text{ abortada} \rangle$: La transacción T_i ha sido abortada.

Cuando una transacción realiza una escritura es fundamental que se cree el registro del registro histórico correspondiente a esa escritura **antes de modificar la base de datos**.

Para que los registros del registro histórico sean útiles para recuperarse frente a errores del disco o del sistema, el registro histórico **debe residir en almacenamiento estable**.

Modificación diferida

La **técnica de la modificación diferida** garantiza la atomicidad de las transacciones mediante el almacenamiento de todas las modificaciones de la base de datos en el registro histórico, pero retardando la ejecución de todas las operaciones *escribir* de una transacción hasta que la transacción se compromete parcialmente.

Cuando una transacción se **compromete parcialmente**, la información del registro histórico asociada a esa transacción se utiliza para la ejecución de las escrituras diferidas. Si el sistema cae antes de que la transacción complete su ejecución o si la transacción aborta, la información del registro histórico simplemente se ignora.

	Registro histórico	Base de datos
T_0 : leer(A)	$\langle T_0 \text{ iniciada} \rangle$	
$A := A - 50$	$\langle T_0, A, 950 \rangle$	
escribir(A)	$\langle T_0, B, 2050 \rangle$	
leer(B)	$\langle T_0 \text{ comprometida} \rangle$	$A = 950$
$B := B + 50$		$B = 2050$
escribir(B)		
T_1 : leer(C)	$\langle T_1 \text{ iniciada} \rangle$	
$C := C - 100$	$\langle T_1, C, 600 \rangle$	
escribir(C)	$\langle T_1 \text{ comprometida} \rangle$	$C = 600$

Estado del registro histórico y de la base de datos correspondiente a T_0 y T_1
(Inicialmente A, B y C contienen 1000, 2000 y 700 respectivamente)

Si el fallo ocurre durante la ejecución de las escrituras diferidas, hay que asegurarse de que, antes del comienzo de esta actualización, todos los registros del registro histórico se guardan en almacenamiento estable. Una vez que se ha hecho esto, la actualización real tiene lugar y la transacción pasa al estado comprometido.

Mediante la utilización del registro histórico, el sistema puede manejar cualquier fallo que conduzca a la pérdida de información en el almacenamiento volátil. El esquema de recuperación usa el siguiente procedimiento de recuperación:

- **rehacer(T_i)**: Fija el valor de todos los elementos de datos actualizados por la transacción T_i a los valores nuevos.

La operación rehacer debe ser **idempotente**, esto es, el resultado de ejecutarla varias veces debe ser equivalente al resultado de ejecutarla una sola vez.

$\langle T_0 \text{ iniciada} \rangle$	$\langle T_0 \text{ iniciada} \rangle$	$\langle T_0 \text{ iniciada} \rangle$
$\langle T_0, A, 950 \rangle$	$\langle T_0, A, 950 \rangle$	$\langle T_0, A, 950 \rangle$
$\langle T_0, B, 2050 \rangle$	$\langle T_0, B, 2050 \rangle$	$\langle T_0, B, 2050 \rangle$
	$\langle T_0 \text{ comprometida} \rangle$	$\langle T_0 \text{ comprometida} \rangle$
	$\langle T_1 \text{ iniciada} \rangle$	$\langle T_1 \text{ iniciada} \rangle$
	$\langle T_1, C, 600 \rangle$	$\langle T_1, C, 600 \rangle$
		$\langle T_0 \text{ comprometida} \rangle$
(a)	(b)	(c)

Suponer que ocurre una caída justo después de escribir en almacenamiento estable el registro del registro histórico para el paso **escribir(B)** (a), **escribir(C)** (b), $\langle T_1 \text{ comprometida} \rangle$ (c).

Después de ocurrir un fallo, el subsistema de recuperación consulta el registro histórico para determinar las transacciones que deben rehacerse. **Una transacción T_i debe rehacerse si y sólo si el registro histórico contiene los registros $\langle T_i \text{ iniciada} \rangle$ y $\langle T_i \text{ comprometida} \rangle$.**

Modificación inmediata

La técnica de la **modificación inmediata** permite realizar la salida de las modificaciones de la base de datos a la propia base de datos mientras que la transacción está todavía en activo. Las modificaciones de datos escritas por transacciones activas se denominan modificaciones no comprometidas.

Como la información del registro histórico se utiliza para reconstruir el estado de la base de datos, la actualización real de la base de datos no puede permitirse antes de que el registro del registro histórico correspondiente se haya escrito en almacenamiento estable. Por lo tanto, es necesario que antes de la ejecución de una

operación $salida(B)$, se escriban en almacenamiento estable los registros del registro histórico correspondientes a B .

	Registro histórico	Base de datos
T_0 : leer(A)	$\langle T_0 \text{ iniciada} \rangle$	
$A := A - 50$	$\langle T_0, A, 1000, 950 \rangle$	
escribir(A)		
leer(B)	$\langle T_0, B, 2000, 2050 \rangle$	
$B := B + 50$		$A = 950$
escribir(B)		$B = 2050$
T_1 : leer(C)	$\langle T_0 \text{ comprometida} \rangle$	
$C := C - 100$	$\langle T_1 \text{ iniciada} \rangle$	
escribir(C)	$\langle T_1, C, 700, 600 \rangle$	
		$C = 600$
	$\langle T_1 \text{ comprometida} \rangle$	

Estado del registro histórico y de la base de datos correspondiente a T_0 y T_1
(Inicialmente A, B y C contienen 1000, 2000 y 700 respectivamente)

Mediante la utilización del registro histórico, el sistema puede manejar cualquier fallo que no genere una pérdida de información en el almacenamiento no volátil. El esquema de recuperación usa dos procedimientos de recuperación:

- **deshacer**(T_i): Restaura el valor de todos los elementos de datos actualizados por la transacción T_i a los valores anteriores.
- **rehacer**(T_i): Fija el valor de todos los elementos de datos actualizados por la transacción T_i a los nuevos valores.

Las operaciones deshacer y rehacer deben ser **idempotentes** para garantizar un comportamiento correcto incluso en el caso de que el fallo se produzca durante el proceso de recuperación.

$\langle T_0 \text{ iniciada} \rangle$	$\langle T_0 \text{ iniciada} \rangle$	$\langle T_0 \text{ iniciada} \rangle$
$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$
$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$
	$\langle T_0 \text{ comprometida} \rangle$	$\langle T_0 \text{ comprometida} \rangle$
	$\langle T_1 \text{ iniciada} \rangle$	$\langle T_1 \text{ iniciada} \rangle$
	$\langle T_1, C, 700, 600 \rangle$	$\langle T_1, C, 700, 600 \rangle$
		$\langle T_1 \text{ comprometida} \rangle$
(a)	(b)	(c)
deshacer (T_0)	deshacer (T_1) rehacer (T_0)	rehacer (T_1) rehacer (T_0)

Suponer que ocurre una caída justo después de escribir en almacenamiento estable el registro del registro histórico para el paso **escribir**(B) (a), **escribir**(C) (b), $\langle T_i \text{ comprometida} \rangle$ (c).

Después de haberse producido un fallo, el esquema de recuperación consulta el registro histórico para determinar las transacciones que deben rehacerse y las que deben deshacerse.

- Una transacción T_i debe **deshacerse** si el registro histórico contiene el registro $\langle T_i \text{ iniciada} \rangle$, pero no contiene el registro $\langle T_i \text{ comprometida} \rangle$.
- Una transacción T_i debe **rehacerse** si el registro histórico contiene los registros $\langle T_i \text{ iniciada} \rangle$ y $\langle T_i \text{ comprometida} \rangle$.

Puntos de revisión

Para reducir el número de registros del registro histórico que deben ser examinados cuando el sistema se recupera de una caída, se introducen los puntos de revisión. Durante la ejecución, el sistema actualiza el registro histórico utilizando una de las dos

técnicas que se han descrito anteriormente. Además, el sistema realiza periódicamente **puntos de revisión**, en los cuales tiene lugar la siguiente secuencia de acciones:

- Escritura en almacenamiento estable de todos los registros del registro histórico que residan en ese momento en memoria principal
- Escritura en disco de todos los bloques de memoria intermedia que se hayan modificado.
- Escritura en almacenamiento estable de un registro del registro histórico *< revisión >*.

Mientras se lleva a cabo un punto de revisión no se permite que ninguna transacción realice acciones de actualización, tales como escribir en un bloque de memoria intermedia o escribir en un registro de registro histórico.

Cuando se produce un fallo, el esquema de recuperación examina el registro histórico para determinar la última transacción T_i que comenzó su ejecución antes de que tuviera lugar el último punto de revisión. Para encontrar una transacción de este tipo se empieza a buscar por el final de registro histórico hasta que se encuentra el primer registro *< revisión >*; después se continúa la búsqueda hacia atrás hasta que se encuentra el siguiente registro *< T_i iniciada >*.

Una vez que ha sido identificada la transacción T_i sólo es necesario aplicar las operaciones *rehacer* y *deshacer* a la transacción T_i y a las transacciones T_j que comenzaron su ejecución después que T_i . El conjunto exacto de operaciones de recuperación que han de llevarse a cabo depende de si se está usando la técnica de modificación inmediata o la de modificación diferida. Si se emplea la técnica de modificación inmediata, las operaciones de recuperación deben ser las siguientes:

- Ejecutar **deshacer**(T_k) para todas las transacciones T_k de T (conjunto de transacciones) para las que no exista un registro *< T_k comprometida >* en el registro histórico.
- Ejecutar **rehacer**(T_k) para todas las transacciones T_k de T para las que aparece un registro *< T_k comprometida >* en el registro histórico.

Obviamente no es necesario aplicar la operación *rehacer* cuando se está utilizando la técnica de modificación diferida.

18. Registro histórico con transacciones concurrentes: retroceso de transacciones. Puntos de revisión. Recuperación al reiniciar.

Retroceso de transacciones

Se utiliza el registro histórico para retroceder una transacción T_i fallida. El registro histórico se explora hacia atrás; para cada registro del registro histórico de la forma *< T_i, X_j, V_1, V_2 >*, se restablece el valor del elemento de datos X_j con su valor anterior: V_1 . La exploración del registro histórico termina cuando se encuentra el registro *< T_i iniciada >*.

Si para el control de concurrencia se utiliza el bloqueo estricto de dos fases, los bloqueos llevados a cabo por una transacción T sólo pueden ser desbloqueados después de que la transacción se haya retrocedido. Una vez que T (que se está retrocediendo) haya actualizado un elemento de datos, ninguna otra transacción podría haber actualizado el mismo elemento de datos debido a los requisitos del

control de concurrencia. Así pues, la restitución del valor anterior de un elemento de datos no borrará los efectos de otra transacción.

Puntos de revisión

Cuando las transacciones pueden ejecutarse concurrentemente, la situación se torna más complicada ya que varias transacciones pueden estar activas en el momento en que se produce el último punto de revisión.

En un sistema de procesamiento de transacciones concurrente es necesario que el registro del registro histórico correspondiente a un punto de revisión sea de la forma $\langle \text{revisión } L \rangle$, donde L es una lista con las transacciones activas en el momento del punto de revisión. Mientras que se realiza el punto de revisión, las transacciones no efectúan modificaciones ni sobre los bloques de memoria intermedia ni sobre el registro histórico.

Un punto de revisión durante el cual se permite que las transacciones realicen modificaciones incluso mientras los bloques de memoria intermedia se están guardando en disco, se denomina **punto de revisión difuso**.

Recuperación al reiniciar

El sistema construye dos listas cuando se recupera de una caída: la *lista – deshacer*, que consta de las transacciones que han de deshacerse, y la *lista – rehacer*, que está formada por las transacciones que deben rehacerse.

Al principio ambas están vacías. Luego se recorre el registro histórico hacia atrás examinando cada registro hasta que se encuentra el primer registro $\langle \text{revisión } \rangle$.

- Para cada registro encontrado de la forma $\langle T_i \text{ comprometida} \rangle$ se añade T_i a la *lista – rehacer*.
- Para cada registro encontrado de la forma $\langle T_i \text{ iniciada} \rangle$, si T_i no está en la *lista – rehacer*, entonces se añade T_i a la *lista – deshacer*.

Cuando se terminan la *lista – rehacer* y la *lista – deshacer*, el proceso de recuperación procede de la siguiente manera:

- a. Se recorre de nuevo el registro histórico **hacia atrás** comenzando en el último registro y se realiza una operación *deshacer* por cada registro del registro histórico que pertenezca a una transacción T_i de la *lista – deshacer*. En esta fase se ignoran los registros del registro histórico concernientes a transacciones de la *lista – rehacer*. El recorrido del registro histórico termina cuando se encuentran registros $\langle T_i \text{ iniciada} \rangle$ para cada transacción T_i de la *lista – deshacer*.
- b. Se localiza el último registro $\langle \text{revisión } L \rangle$ del registro histórico. Nótese que este paso puede necesitar de un recorrido del registro histórico hacia delante si el registro punto de revisión quedó atrás en el paso **a**.
- c. Se recorre el registro histórico **hacia delante** desde el último registro $\langle \text{revisión } L \rangle$ y se realiza una operación *rehacer* por cada registro del registro histórico que pertenezca a una transacción T_i de la *lista – rehacer*. En esta fase se ignoran los registros del registro histórico concernientes a transacciones de la *lista – deshacer*.

Cuando se ha completado el proceso de recuperación se continúa con el procesamiento normal de las transacciones.

Es importante el hecho de deshacer las transacciones de la *lista – deshacer* antes de rehacer las transacciones de la *lista – rehacer* al utilizar los pasos a, b y c del algoritmo anterior.

19. Fallo con pérdida de almacenamiento no volátil: Volcado. Recuperación. Coste.

La idea básica es **volcar** periódicamente el contenido entero de la base de datos en almacenamiento estable. Se utilizará el volcado más reciente para que la base de datos recupere un estado consistente cuando ocurra un fallo que conduzca a la pérdida de algunos bloques físicos de la base de datos. Una vez que se complete esta operación, el sistema utilizará el registro histórico para llevar al sistema de base de datos al último estado consistente en el que estuvo antes de producirse el fallo.

Ninguna transacción puede estar activa durante el procedimiento de **volcado** y tendrá lugar una secuencia de acciones similar a la utilizada en los puntos de revisión:

- Escribir en almacenamiento estable todos los registros del registro histórico que residan en ese momento en memoria principal.
- Escribir en disco todos los bloques de la memoria intermedia.
- Copiar en el contenido de la base de datos en almacenamiento estable.
- Escribir el registro del registro histórico *< revisión >* en almacenamiento estable.

Para la **recuperación** por pérdida de almacenamiento no volátil se restituye la base de datos en el disco utilizando el último volcado realizado. Entonces se consulta el registro histórico y se rehacen todas las transacciones que se hubieran comprometido desde que se efectuó ese último volcado. Nótese que no es necesario ejecutar ninguna operación deshacer.

Un volcado del contenido de una base de datos se denomina también **volcado de archivo** ya que pueden archivarse los volcados y utilizarlos más tarde para examinar estados anteriores de la base de datos.

El procedimiento de volcado descrito es **costoso** debido a las dos siguientes razones:

- Debe copiarse en almacenamiento estable la base de datos entera, lo que conlleva una considerable transferencia de datos.
- Se pierden ciclos de UCP porque se detiene el procesamiento de transacciones durante el procedimiento de volcado.

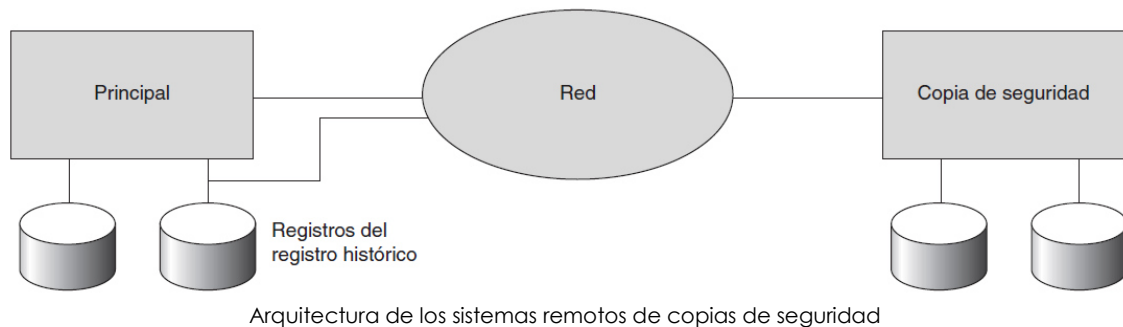
Se han desarrollado esquemas de **volcado difuso** que permiten que las transacciones sigan activas mientras se realiza el volcado.

20. Sistemas remotos de copia de seguridad: funcionamiento. Detección de los fallos. Transferencia de control. Tiempo de recuperación. Tiempo de compromiso.

Hay una necesidad creciente de sistemas de procesamiento de transacciones que ofrezcan una disponibilidad elevada y que puedan funcionar pese a los desastres ambientales. Estos sistemas deben proporcionar una **disponibilidad elevada**, es decir, **el tiempo en que el sistema no es utilizable debe ser extremadamente pequeño**.

Se puede obtener una disponibilidad elevada realizando el procesamiento de transacciones en un solo sitio, denominado **sitio principal**, pero tener un **sitio remoto copia de seguridad**, en el que se repliquen todos los datos del sitio principal. El sitio remoto copia de seguridad se denomina también a veces **sitio secundario**. La

sincronización entre los dos sitios se obtiene enviando todos los registros del registro histórico desde el sitio principal al sitio remoto copia de seguridad.



Cuando falla el sitio principal, el sitio remoto copia de seguridad asume el procesamiento. Llevará a cabo, en primer lugar, la **recuperación** utilizando su copia (tal vez anticuada) de los datos del sitio principal y los registros del registro histórico recibidos del mismo. Una vez realizada la recuperación, el sitio remoto copia de seguridad comienza a procesar transacciones.

Varios aspectos que deben abordarse al diseñar sistemas remotos para copias de seguridad son: **detección de fallos**, **transferencia del control**, **tiempo de recuperación** y **tiempo de compromiso**.

Detección de fallos

Es importante que el sistema remoto de copia de seguridad detecte que el sitio principal ha fallado. El fallo de las líneas de comunicación puede hacer creer al sitio remoto copia de seguridad que el sitio principal ha fallado. Para evitar este problema hay que **mantener varios enlaces de comunicaciones** con modos de fallo independientes entre el sitio principal y el sitio remoto copia de seguridad.

Transferencia de control

Cuando el sitio principal falla, el sitio copia de seguridad asume el procesamiento y se transforma en el nuevo sitio principal. La manera más sencilla de **transferir el control** es que el sitio principal antiguo reciba el registro histórico de operaciones rehacer del sitio copia de seguridad antiguo y se ponga al día con las actualizaciones aplicándolas de manera local. El sitio principal antiguo puede entonces actuar como sitio remoto copia de seguridad. Si hay que devolver el control, el sitio remoto copia de seguridad antiguo puede simular que ha fallado, lo que da lugar a que el sitio principal antiguo asuma el control.

Tiempo de recuperación

El sitio remoto copia de seguridad puede procesar de manera periódica los registros rehacer del registro histórico que haya recibido y realizar un punto de revisión de manera que se puedan borrar las partes más antiguas del registro histórico. Como consecuencia se puede reducir el retraso antes de que el sitio remoto copia de seguridad asuma el control.

En una configuración de **relevo en caliente** el sitio remoto copia de seguridad procesa los registros rehacer del registro histórico según llegan, y aplica las actualizaciones de forma local. Tan pronto como se detecte el fallo del sitio principal, el sitio copia de seguridad completa la recuperación retrocediendo las transacciones incompletas y queda preparado para procesar las nuevas.

Tiempo de compromiso

Para asegurar que las actualizaciones de una transacción comprometida sean duraderas no se deben declarar comprometida una transacción hasta que sus **registros del registro histórico hayan alcanzado el sitio copia de seguridad**.

Los grados de durabilidad pueden clasificarse de la manera siguiente:

- **Uno seguro:** Las transacciones se comprometen tan pronto como sus registros de compromiso del registro histórico se escriben en un almacenamiento estable en el sitio principal. Puede que se necesite la intervención humana para devolver a la base de datos a un estado consistente.
- **Dos muy seguro:** Las transacciones se comprometen tan pronto como sus registros de compromiso del registro histórico se escriben en un almacenamiento estable en el sitio principal y en el sitio copia de seguridad. La disponibilidad es realmente menor que en el caso de un solo sitio, aunque la posibilidad de la pérdida de datos es mucho más reducida.
- **Dos seguro:** Este esquema es idéntico al esquema dos muy seguro si tanto el sitio principal como el sitio copia de seguridad están activos. Si sólo está activo el sitio principal se permite que la transacción se comprometa tan pronto como su registro de compromiso del registro se escriba en un almacenamiento estable en el sitio principal. Este esquema proporciona mejor disponibilidad que el esquema dos muy seguro al tiempo que evita el problema de las transacciones perdidas afrontado por el esquema uno seguro.

Seguridad

21. Privilegios. Concesión de los privilegios. Concesión en SQL.

Se pueden asignar a los usuarios varios tipos de privilegios recogidos por la norma SQL para diferentes partes de la base de datos (lectura/**select**, inserción/**insertn**, actualización/**update** y borrado/**delete**) mediante la orden **grant** <privilegios> **on** <relacion o vista> **to** <usuario o rol>.

SQL también soporta privilegios como los de crear, borrar o modificar relaciones y ejecutar procedimientos. **All privileges** es una forma abreviada de todos los privilegios que se pueden conceder. Puede permitirse a un usuario que trasmita un privilegio a otros (**with grant option**).

El nombre de usuario **public** hace referencia a todos los usuarios actuales y futuros de la base de datos.

Para retirar privilegios se usa la orden **revoke** <lista de privilegios> **on** <relacion o vista> **from** <usuarios o roles>.

Sólo el propietario del esquema puede realizar modificaciones sobre el mismo. SQL también incluye un privilegio **references** que permite crear claves externas (que restringen operaciones de borrado y actualización), o **execute**, que permite ejecutar funciones o procedimientos, y el **usage**, que permite a otros usuarios emplear el dominio especificado. La forma de autoridad definitiva es el administrador de la base de datos. Los privilegios **update** e **insert** pueden concederse sobre todos los atributos de una relación o sólo sobre algunos, mientras que **references** especifica siempre atributos concretos.

22. Roles. Cancelación de autorizaciones en SQL. Autorizaciones en vistas, funciones y procedimientos.

Roles

Un rol es un papel o una función que se desenvuelve. En la base de datos se crea un conjunto de roles a los que se le conceden autorizaciones del mismo modo que a los usuarios. Cada usuario de la base de datos está autorizado a desenvolver un conjunto de roles.

El empleo de los roles tiene la ventaja de exigir a los usuarios que se conecten con su propio identificador de usuario.

Puede haber cadena de roles. Los privilegios concedidos a un usuario o rol consisten en todos los privilegios concedidos directamente, y todos los privilegios concedidos al rol desenvuelto por el usuario o rol.

Las acciones ejecutadas por las sesiones tienen todos los privilegios concedidos directamente al usuario que las ha atribuido, así como todos los privilegios concedidos a los roles que se le concediesen.

Cancelación de autorizaciones en SQL.

La transmisión de una autorización de un usuario a otro puede representarse mediante el grafo de autorización, cuyos nodos son los usuarios. Existe arco $U_i \rightarrow U_j$ si U_i le ha transmitido un permiso a U_j . La raíz del grafo es el administrador de la base de

datos y un usuario tiene una autorización si e sólo si existe un camino desde la raíz del grafo.

Para retirar autorizaciones se emplea la orden **revoke** <privilegios> **on** <relaciones o vistas> **from** <usuarios o roles> [**restrict** | **cascade**]. La retirada de privilegios puede provocar que otros usuarios o roles también pierdan los suyos (retirada en cascada). En el caso de especificarse la opción **restrict**, el sistema devuelve error si hay retiradas en cascada. Para abordar esto, SQL 1999 permite sólo a los roles conceder privilegios.

Autorizaciones en vistas, funciones y procedimientos

Aún que se le pueden negar a un usuario el acceso a una relación, se le puede permitir acceder a parte de ella mediante una vista. Cuando un usuario haga una consulta sobre esa parte da información, el procesador la traducirá a una consulta con relaciones reales, por lo que el sistema deberá comprobar la autorización de la consulta del usuario antes de procesarla. El usuario que crea la vista no tiene por qué tener todos los privilegios sobre ella, sólo tendrá aquellos que no le den ninguna autorización adicional respecto de las que ya tenía.

El privilegio **execute** se puede conceder sobre una función o procedimiento y permite que el usuario los ejecute. Las funciones y procedimientos tienen los mismos privilegios que su creador, y se ejecutan como si quien los invocase fuese siempre el usuario que los creó.

23. Limitaciones de autorización en SQL. Trazas de auditoría.

Limitaciones de autorización en SQL

Las normas SQL para la autorización tienen algunas deficiencias, como no permitir la autorización a nivel de tuplas. Con el crecimiento de la web, es posible que los usuarios finales no tengan identificadores de usuario individuales. La autorización recae entonces sobre el servidor de aplicaciones, que en algunos casos puede implementar autorizaciones más pormenorizadas. El código para comprobar la autorización se mezcla con el resto del código, por lo que se hace más difícil garantizar la ausencia de agujeros de seguridad.

Una forma de restringir las tuplas de acceso es añadir una cláusula **where** donde se compruebe si el **id** de usuario coincide con el registrado en la base de datos, aunque esto no es muy buena idea ya que se quiere hacer un acceso global, no será posible.

Trazas de auditoría

Una traza de auditoría es un registro de todas las modificaciones de la base de datos, junto con información acerca del usuario que las realizó y en qué momento lo hizo. Esto mejora la seguridad, ya que se puede seguir la pista de todas las actualizaciones realizadas por los usuarios para encontrar actualizaciones incorrectas o fraudulentas.

Es posible crear trazas de auditoría definiendo disparadores, pero muchos sistemas de bases de datos proporcionan mecanismos específicos más cómodos de usar.

Implementación

24. Organización de los archivos: Registros de tamaño fijo y variable.

Los archivos se organizan como secuencias de registros. Estos registros se corresponden con los bloques del disco. Aunque los bloques son de un tamaño fijo determinado por las propiedades físicas del disco y por el sistema operativo, los tamaños de los registros varían.

Registros de tamaño fijo

Supongamos que tenemos un tipo de dato definido con una longitud de 40 bytes. Un enfoque sencillo es emplear los primeros 40 bytes para un registro, los 40 siguientes para un segundo, y así sucesivamente. Pero este enfoque presenta dos problemas:

- Resulta difícil borrar registros, ya que hay que llenar el espacio vacío.
- A menos que el tamaño de los bloques sea múltiplo de 40, algún registro saltará los límites de los bloques. Habrá que leer más de un bloque para acceder a los registros.

Cuando se borra un registro, la opción más sencilla es desplazar el último registro del archivo al espacio que quedó libre. No resulta deseable desplazar bloques, ya que implica más accesos a bloques y, dado que la inserción es más frecuente que el borrado, resulta aceptable dejar libre el espacio y esperar a que lo ocupe una inserción posterior. Para esto, al principio del archivo se le asigna una **cabecera de archivo** que contendrá la dirección del primer registro borrado, de forma que los registros borrados contengan un puntero al registro siguiente, formando una lista enlazada denominada **lista libre**. Si no hay espacio disponible, se añade el nuevo registro al final del archivo.

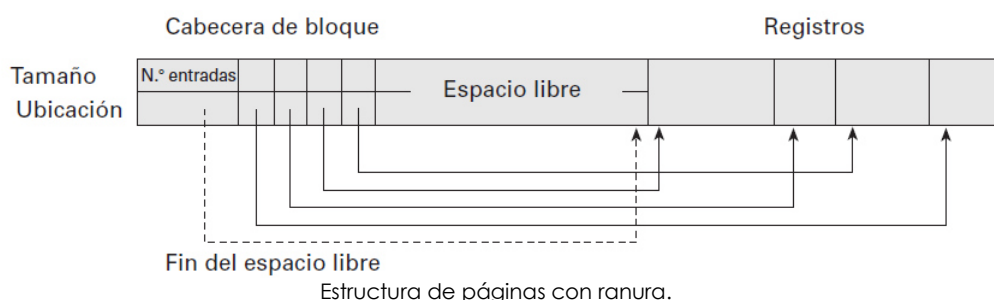
Registros de tamaño variable

Los registros de tamaño variable surgen en los sistemas de bases de datos por:

- Almacenamiento de varios tipos de registros en un mismo archivo.
- Tipos de registro que permiten longitudes variables para uno o varios de los campos.
- Tipos de registro que permiten campos repetidos.

Se implementan habitualmente mediante la **estructura de páginas con ranuras**, donde hay:

- Una cabecera al principio de cada bloque que contiene el número de elementos del registro.
- El final del espacio vacío del bloque.
- Un array cuyas entradas contienen la localización y tamaño de cada registro.

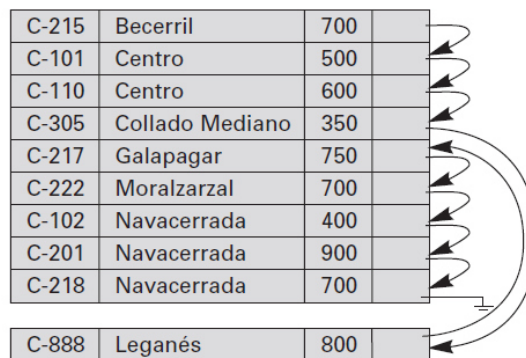


Los registros se localizan en el bloque de forma contigua, empezando por el final. El espacio libre dentro del bloque es contiguo, entre la última entrada del array y el primer registro. Si se inserta un registro, se le asigna espacio al final del espacio libre y se añade la entrada correspondiente a la cabecera. Si se borra un registro, se desplazan los registros del bloque situados antes que él. El coste no es demasiado elevado ya que el tamaño del bloque es limitado.

25. Organización dos registros en archivos: archivos secuenciales. Archivos en agrupaciones.

Archivos secuenciales

Los archivos secuenciales están diseñados para el procesamiento eficiente de los registros de acuerdo con un orden basado en **una clave de búsqueda**. Una clave de búsqueda es cualquier atributo o conjunto de atributos; no tiene por qué ser una clave primaria ni una superclave. Para permitir la recuperación rápida de los registros según la clave de búsqueda, estos se vinculan mediante punteros que señalan el siguiente registros según el orden indicado por la clave de búsqueda o lo más cercano posible.



Archivo secuencial después de una inserción

No obstante, resulta difícil mantener el orden físico secuencial cuando se insertan y borran registros, dado que resulta costoso desplazar muchos registros como consecuencia de una sola inserción o borrado. Se pueden gestionar el borrado utilizando cadenas de punteros. Para la inserción se aplican las reglas siguientes:

- Se localiza la clave del archivo que precede al registro a insertar según el orden de la clave de búsqueda.
- Si hay algún registro vacío dentro del mismo bloque, se inserta en él, en caso contrario, se inserta en un bloque de desbordamiento. En cualquier caso, hay que ajustar los punteros para vincular los registros según el orden de la clave de búsqueda.

La correspondencia entre el orden de clave de búsqueda y la física pueden perderse totalmente, por lo que procesamiento secuencial resulta ineficiente; en estos casos se debe reorganizar el archivo, lo que resulta una operación compleja y que se debe realizar en momentos en los que la carga de trabajo del sistema sea baja.

Organización de archivos en agrupaciones

Una organización de archivos en agrupaciones es una organización de archivos que almacenan registros relacionados de dos o más relaciones de cada bloque. Esta organización permite leer muchos de los registros que satisfacen la condición de

reunión utilizando un solo proceso de lectura de bloques. Por tanto, se puede procesar esta consulta concreta de manera más eficiente.

Este sistema produce retardo en el procesamiento de consultas simples. La determinación del momento de utilizar la agrupación depende de los tipos de consulta que el diseñador de la base de datos considere más frecuentes.

26. Árboles B+: Descripción. Estructura. Consulta. Organización de archivos con árboles B+.

Descripción

Un índice de árbol B+ toma forma de un árbol equilibrado donde los caminos de la raíz a cada hoja son de la misma longitud. Cada nodo interno tiene entre $n/2$ y n hijos, donde n es fija para cada árbol concreto. Esta estructura implica una degradación en el rendimiento de inserción y borrado y un espacio extra, pero es justificable ya que evita el coste de reorganizar el archivo.

Estructura

En los nodos hoja, los punteros apuntan ya bien a un registro del archivo con valor de la clave de búsqueda, o bien a un cajón de punteros. La estructura cajón sólo se emplea si la clave de búsqueda no es una clave candidata y el archivo no está ordenado. Los nodos internos tienen una estructura similar, pero sus punteros apuntan a otros nodos del árbol.

El nodo raíz puede tener menos de $n/2$ punteros, pero debe tener un mínimo de dos a no ser que se trate de un árbol de un solo nodo.

Consulta

Supongamos que queremos encontrar un valor V . Se examina el nodo raíz para buscar el menor valor de clave de búsqueda mayor o igual que V , y se sigue el puntero. Si no hay dicho número, se sigue al último puntero. Se realiza el mismo procedimiento hasta llegar a un nodo hoja donde estará la clave de búsqueda igual a V cuyo puntero nos llevará al registro o cajón deseado. En caso de no encontrar en el nodo hoja una clave de búsqueda igual a V , es que no existe.

Organización de archivos con árboles B+

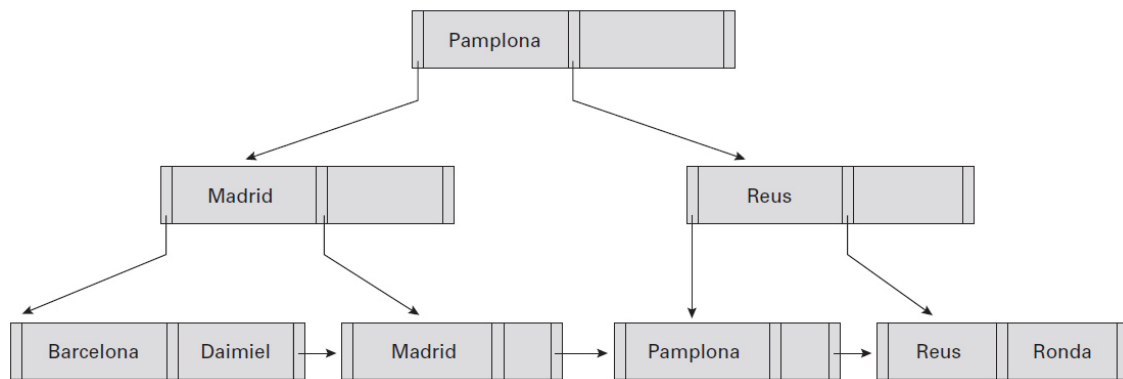
En la organización de archivos con árboles B+, los nodos hoja contienen registros, en lugar de punteros a registros. Como los registros suelen ser más grandes que los punteros, el número máximo de registros que se pueden almacenar en un nodo hoja es menor que el número de punteros en un nodo interno. La inserción y borrado se tratan de la misma forma que con los índices de árboles B+.

Cuando se inserta V , se localiza el bloque en el que debería estar mediante la búsqueda en el árbol de mayor clave o igual que V . Si el bloque localizado tiene suficiente espacio, V se inserta en él. En caso contrario se divide el bloque en dos y se redistribuyen sus registros. Esta división se propaga hacia arriba.

Cuando se borra V , se eliminan de su bloque y, si como resultado éste pasa a ocupar menos de la mitad, sus registros se redistribuyen con los registros del bloque adyacente.

Cuando se emplean árboles B+ para la organización de un archivo, la utilización de espacio es importante ya que los registros ocupan más que las claves y los punteros.

Esto puede mejorarse implicando a más nodos hermanos en la redistribución mediante divisiones y fusiones. En la inserción, si un nodo está lleno y los nodos adyacentes también lo están, se divide el nodo y las entradas se reparten entre un nodo adyacente y los dos nuevos nodos, de forma que cada uno tendrá como mínimo $2n/3$ entradas. En el borrado, si un nodo tiene menos de $2n/3$, se intenta tomar prestada una entrada de un nodo hermano. Si no es posible, se redistribuyen las entradas y se vacían y elimina uno de los nodos. La actualización se vuelve más costosa a medida que hay más nodos hermanos implicados.

Ejemplo árbol B⁺ para n=3

27. Asociación estática.

La organización de archivos basa en la técnica de hashing permite evitar el acceso a la estructura de índice, al mismo tiempo que proporciona una forma de construirlos. Para insertar un registro con una clave de búsqueda K, se calcula $h(K)$ para obtener la dirección del cajón de ese registro. Para buscar la clave de búsqueda K, se calcula $h(K)$ y se comprueban todos los registros del cajón para asegurarse de que el registro es el deseado. El hashing puede emplearse para la organización de archivo asociativo, donde se obtiene directamente la dirección de bloque de disco que contiene el registro deseado calculando una función del valor de la clave de búsqueda del registro. En la organización de índice asociativo se organizan las claves de búsqueda con sus punteros asociados en una estructura de archivo asociativo.

Una función de hashing es ideal si distribuye las claves almacenadas uniformemente entre todos los cajones. La distribución deberá tener las siguientes propiedades:

- **Distribución uniforme:** Se asignan a cada cajón el mismo número de valores de clave de búsqueda dentro de un conjunto de todos los valores posibles de la misma.
- **Distribución aleatoria:** En el caso medio, cada cajón tendrá casi el mismo número de valores. El valor de asociación no estará correlacionado con ningún ordenamiento de los valores de clave de búsqueda visible exteriormente.

Si el cajón no tiene suficiente espacio, se produce desbordamiento. Puede producirse por un número de cajones insuficiente o por atasco (si hay varios registros con la misma clave o si la función genera una distribución no uniforme). Para evitar el desbordamiento, se introduce un factor de corrección, que implica una pérdida de espacio.

El desbordamiento se trata con cajones de desbordamiento donde se insertará los registros cuyo registro de destino esté lleno. Todos los cajones de desbordamiento del mismo cajón están enlazados por una lista enlazada (cadena de desbordamiento).

Para tratar las cadenas de desbordamiento hay que modificar el algoritmo de búsqueda para que busque también en los cajones de desbordamiento (de haberlos). Esta implementación se denomina asociación cerrada.

En la asociación abierta, si un cajón está lleno, el sistema inserta los registros en algún otro cajón (por ejemplo el siguiente). Presenta problemas para el borrado.

28. Asociación dinámica.

Algunas técnicas de asociación dinámica permiten modificar dinámicamente la función hash. La asociación extensible hace frente a los cambios en la base de datos dividiendo y fusionando cajones a medida que la base de datos aumenta o disminuye. No se crea un cajón para cada valor de función hash, sino que se crean bajo demanda.

Para localizar el cajón que contiene una clave de búsqueda K , se toman los bits más significativos de $h(K)$, se busca en la entrada de la tabla correspondiente a esa cadena de bits y se sigue el puntero del cajón de esa entrada en la tabla.

Para insertar un registro, se sigue el mismo procedimiento hasta que se llega a un cajón donde se inserta si hay espacio. De no haberlo, se divide el cajón y se redistribuyen los registros. Hay que determinar si es necesario aumentar el número de bits a emplear:

- Si el número de bits es igual al número de bits del cajón, sólo apunta al cajón una entrada de la tabla de direcciones y es necesario incrementar el tamaño de la tabla considerando un bit más de la función hash.
- Si el número de bits es mayor que el del cajón, más de una entrada apunta al cajón. Se puede dividir sin aumentar el tamaño de la tabla. Se recalcula la función y se reintenta la inserción. Para borrar un registro, se busca y se borra. Se pueden fusionar cajones.

29. Comparación entre indexación y asociación

Para hacer una buena elección de organización de archivos y de técnica de indexado hay que tener en cuenta el coste de reorganización periódica del índice o estructura asociativa, la frecuencia de las inserciones y borrados, el tiempo medio de acceso y el tipo de consultas que se supone que se van a formular.

Para consultas con una igualdad en la cláusula **where** se prefiere un esquema asociativo, donde el tiempo de búsqueda es independiente del tamaño de la base de datos. La ventaja de los índices es que en el peor caso el gasto temporal es menor.

Las técnicas de índices ordenados son preferibles cuando una consulta especifica un rango de valores ya que, en cuanto se localiza un cajón, se sigue la cadena del puntero índice para leer el siguiente, y así sucesivamente.

Normalmente, se emplea la indexación ordenada, a no ser que se sepa de antemano que las consultas de rango van a ser poco frecuentes. Las organizaciones asociativas son útiles para los archivos temporales creados durante el procesamiento de las consultas.